

References in C++

When a variable is declared as a reference, it becomes an alternative name for an existing variable. A variable can be declared as a reference by putting & in the declaration. Reference variable is another name for an already existing variable. Once a Reference is initialized with a variable, either a variable name or the reference name may be used to refer to the variable.

Creating Reference in C++

Think of a variable name as a label attached to the variable's location in memory. You can then think of a reference as a second label attached to that memory location.

Therefore you can access the contents of the variable through either the original variable name or the reference.

For example suppose we have following example -

```
int i = 17;
```

we can declare reference variable for i as follows

```
int& r = i;
```


Read the & in these declarations as reference
Thus read the first declaration as r is an ^②
integer reference initialized to i and read the
second declaration as s is a double reference
initialized to d.

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
// declare simple variables
```

```
int i;
```

```
double d;
```

```
// declare reference variables
```

```
int &r = i;
```

```
double &s = d;
```

```
i = 5;
```

```
cout << "value of i : " << i << endl;
```

```
cout << "value of i reference : " << r << endl;
```

```
d = 11.7;
```

```
cout << "value of d : " << d << endl;
```

```
cout << "value of d reference : " << s << endl;
```

```
return 0;
```

```
}
```

Output

Value of i : 5

Value of reference : 5

Value of d : 11.7

Value of d reference : 11.7

Ex 2

(3)

```
string food = "Pizza";  
string meal = food;  
cout << food << "\n";  
cout << meal << "\n";
```

Output

Pizza

Pizza

Reference Vs Pointers

after confused with pointers but three major difference b/w references & pointers are.

- You cannot have null reference. You must always be able to assume that a reference is connected to a legitimate piece of storage.
- Once a reference is initialized to an object, it cannot be changed to refer to another object. Pointers can be pointed to another object at any time.
- A reference must be initialized when it is created. Pointers can be initialized at any time.

Memory address

④

In the example of reference & operator was used to create reference variable. But it can also be used to get the memory address of a variable; which is the location of where the variable is stored on the computer.

When a variable is created in C++, a memory address is assigned to the variable. And when we assign a value to the variable, it is stored in this memory address.

To access it use the & operator & the result will represent where the variable is stored.

ex

```
string food = "Pizza";  
cout << &food;           // outputs 0x60fe04
```

note memory address is in hexadecimal form (0x...)

Reference and pointers are important in C++ because they give you the ability to manipulate the data in computer's memory - which can reduce the code and improve the performance.

These 2 features makes C++ stand out from other prog. languages like python & java.

C++ pointers

(5)

we can get memory address of a variable using the & operator.

ex

```
string food = "Pizza"; // food variable of type string
cout << food; // outputs value of food (Pizza)
cout << &food; // outputs memory address of food (0x60fcd4)
```

A pointer however is a variable that stores the memory address as its value.

A pointer variable points to a data type (like int or string) of the same type and is created with the * operator. The address of variable you are working with is assigned to the pointer.

example

```
int main () {
    string food = "Pizza"; // A string variable
    string* ptr = &food; // A pointer variable that stores the address of food

    cout << food << "\n"; // outputs value of food
    cout << &food << "\n"; // outputs memory of food
    cout << ptr << "\n"; // outputs memory address of food with pointer
    return 0;
}
```

~~Exam~~ Three ways to declare pointer variables (6)

string* mystring; // preferred

string *mystring;

string * mystring;

Deferencing

C++ Dereference

Get memory address and Value

In the example from previous we used the pointer variable to get memory address of a variable (used together with the & reference operator). However you can also use the pointer to get the value of the variable, by using the * operator (the dereference operator):

example

string food = "Pizza"; // variable declaration

string* ptr = &food; // pointer declaration

cout << ptr << "\n"; // Reference: output the memory address of food with pointer 0x6cdfe4

cout << *ptr << "\n";

// Dereference the output value of food with pointer (Pizza)

- * In call by value exact value get passed.
- * If we would like to change the passed values of variable, call by reference is used.
- * Here the parameter is passed as reference.
- * Works same as call / pass by address in C, but there * is used, & here & is used.
- * Any modification made in formal parameter is also reflected in actual parameter.

```
void main()
{
    int swap (int, int);
    int a=10;
    int b=20;
    swap (a,b);
    cout << a << b;
}

swap (int &x, int &y)
{
    int temp;
    temp = x;
    x = y;
    y = temp;
}
```

Output

20 10

void add (int, int); // this is our prototype
 when we write any funcⁿ when we pass an argument
 when we are writing add funcⁿ & ~~calling~~ passing 2 int.
~~value~~ variables (value)

void add (int*, int*); (pointer)

↳ passing 2 pointers of integer)

while making function we pass values or variables
 as an argument it is called call by value.

while making function we pass pointer or address
 as an argument it is called call by reference.

Difference b/w call by value & Reference

when we call a funcⁿ using call by value then the
 copy of variable is passed to the function.

So, if there is any change to the variable in that
 funcⁿ then the original value is not changed.

when we call a funcⁿ - - - - call by reference then the
 original value is passed (beoz we are passing the address
 to the funcⁿ & hence if any change is there
 it changes the original value as well.

call by Value

```
#include <stdio.h>
```

```
void change (int);
```

```
void main()
```

```
{ int a;
```

```
printf ("enter a no.");
```

```
scanf ("%d", &a);
```

```
printf ("value of a before  
funcn call = %d", a);
```

```
change (a);  
printf ("value of a after funcn call = %d", a);
```

```
void change (int a)
```

```
{
```

```
a = a + 10;
```

```
printf ("value of a = %d", a);
```

Output

value of a before funcⁿ call = 5

value of a = 15

value of a after funcⁿ call = 5



Call by reference ex

if add of a[5] is 100
↑
p

(C)

```
#include <stdio.h>
void change (int *);
void main()
{ int a, *p;
  printf ("enter no");
  scanf ("%d", &a);
  p = &a;
  printf ("value of a before funcn call = %d", a);
  change (p);
  printf ("value of a after funcn call = %d", a);
}

void change (int *p)
{
  *p = *p + 10 // due to passing add. orig. value is changed.
  printf ("value of a = %d", *p);
}
```

Output

value of a before funcⁿ call = 5

value of a = 15

value of a after funcⁿ call = 15

Pointer

Pointers are used to store address of variable
are used to create runtime memory

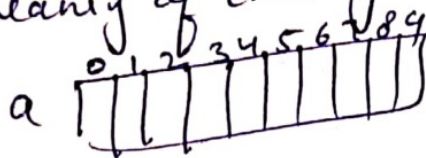
ex

```
int a;  
a = 5
```

a[5]

a memory location may be
having some address
hexadecimal digit

meaning of creating run time memory.



int a[10]

↑ ~~the~~ element
if want to inc. ~~array~~ size not possible
if using less, not used are wasted

So how we can set our array at run time. so that
much of memory will be allocated.

This we do through pointers.

```
#include <stdio.h>
```

```
int main()
```

```
{ int a, b;
```

```
  a = 5;
```

```
  b = 10;
```

```
  printf("address of a = %x", &a);
```

```
  printf("address of b = %x", &b);
```

```
  printf("value of a = %d and b = %d", a, b);
```

```
  return (0);
```

```
}
```

%x → for hexadecimal

add 2 no. using pointers → stores address

include <stdio.h>

int main()

{ int a, b; *p, *q; // as many if that many pointer

printf("enter 2 no.");

scanf("%d %d", &a, &b);

p = &a;

q = &b;

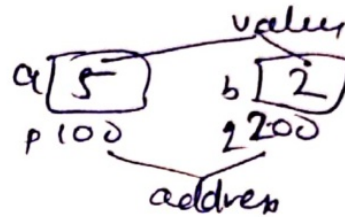
// add. in DS

c = *p + *q;

printf("val = %d", c);

return 0;

}



*p → value at p
*q → value at q
 $5 + 2 = 7$

Call by Value

```
main()
{
    int a=3, b=4;
    f1(a, b);
}
```

↑

formal arguments
void f1(int x, int y)
ordinary variable

// passing value of a & b in x & y
x & y will have copy of a & b
so changes in x & y will
not affect original value

// call a function & pass address

Call by address

```
main()
{
    a 3
    b 4
```

```
int a=3, b=4;
f2(&a, &b);
```

↑

formal argument
pointers

void f2(int *p, int *q)

↑

Since we have formal argument we will pass address in

f2

if we put 5 & 6 in p & q then it will be changed in a & b

Pointer Variable & Ref. Var. both are almost same both contain address in pointer its called addr. in Ref its called Ref

Call by Reference

```
main()
{
    int a=3, b=4;
    f3(a, b);
}
```

↑

```
int a=3, b=4;
f3(a, b);
```

↑

actually not passing value of a & b, its passing addresses because in defn f3 we have ref variable in formal argument

void f3(int &a, int &b)

main function looking same like call by value but here we

2

Modify the Pointer Value

(9)

```
int main() {
```

```
    string food = "pizza";
```

```
    string *ptr = &food;
```

```
    cout << food << "\n"; // outputs value of food.
```

```
    cout << &food; // outputs address value of food.
```

```
    cout << *ptr << "\n"; // Access memory add. of food & outputs its value
```

```
    *ptr = "Hamburger";
```

```
// changes value of pointer
```

```
    cout << *ptr;
```

```
// outputs new value of pointer
```

```
    cout << food << "\n";
```

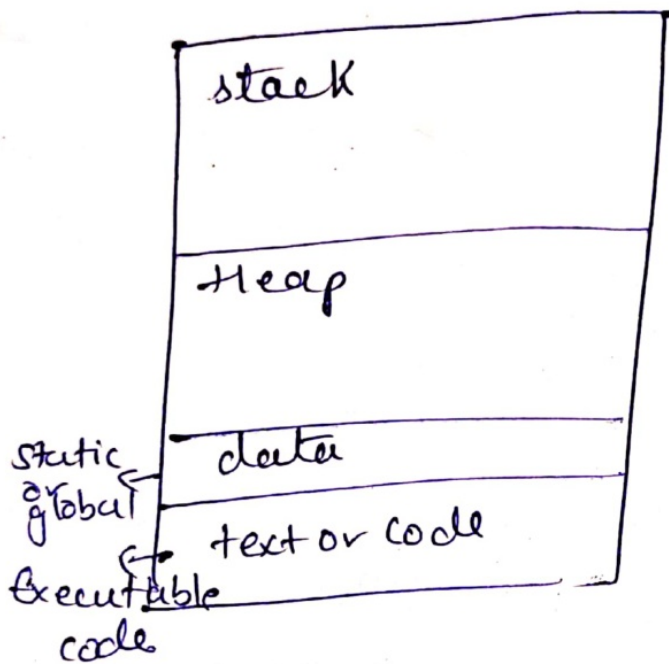
```
// outputs new value of food variable.
```

```
    return 0;
```

```
}
```

Dynamic Memory allocation

dynamic memory allocation in C/C++ refers to performing memory allocation manually by a programmer. Dynamically allocated memory is allocated on heap, and non static and local variables get memory allocated on stack.



```
int e;  
void func()  
{  
    static int a;  
    int b, c;  
    ...  
}
```

```
int main()  
{  
    int d;  
    func();  
    int *ptr = new int[5]  
    return 0;  
}
```

new keyword is used for
DMA

data section has data which has lifetime throughout the program variables b & c are only valid in func() but static variable a & global variable e are available in whole program so it is in data section.

Stack used to store funcⁿ call activation record.

many times you are not aware in advance how much memory you will need to store particular information in a defined variable & the size of required memory can be determined at run time.

You can allocate memory at run time within the heap for the variable of a given type using a special operator in C++ which returns the address of space allocated. This operator is called new operator.

If you are not in need of dynamically allocated memory anymore, you can use delete operator which de-allocates memory that was previously allocated by new operator.

new & delete operators

Syntax of new operator

new datatype;

Here datatype could be any built in data type including an array or any user defined data types include class or structure. Let us start with built in data types. For example we can define a pointer to type double and then request that the memory be allocated at execution time. we do this by following:

double * pvalue = Null; // pointer initialized with null
pvalue = new double; // Request memory for variable.

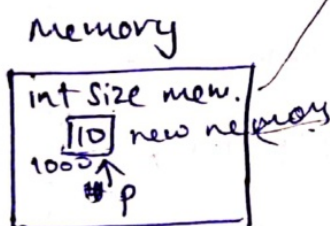
At any point when you feel a variable that has been dynamically allocated is not anymore required, you can free up the memories that in occupied in the free store with delete operator as follows.

delete pvalue; // Release memory pointed to by pvalue

New operator new creates a memory and returns memory address to the pointer.

Syntax new datatype;

Example int *p; // int type pointer is created to hold the address of the data
~~p = new int;~~
 p = new int; // p points to int type of memory



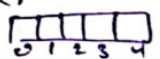
*p = 10;

cout << *p;

ex we can also create an array

int *p

p = new int[5] // p points to 5 locations of int type.



```
#include <iostream>
using namespace std;
int main()
{ int *p;
  p = new int[5];
  p[0] = 10;
  p[1] = 20;
  p[2] = 30;
  p[3] = 40;
  p[4] = 50;
```

```
for (i=0; i<5; i++)
{ cout << p[i] ;
  }
return 0;
```


Delete operator

(13)

↓

deallocate memory

Syntax delete pointername;

eg. delete P;

for arrays- delete [] P;

Program

```
#include <iostream>
using namespace std;
int main()
```

```
{ int *P;
```

```
  P = new int;
```

```
  *P = 500;
```

```
  cout << "values in P" << *P;
```

```
  delete P;
```

```
  cout << "value after delete" << *P; //outputs Garbage Value
```

```
  return 0;
```

```
}
```

Output

values in P 500

value after delete 1234

Garbage Value

for arrays

```
#include <iostream>
using namespace std;
```

```
int main()
```

```
{ int *p;
```

```
  p = new int[2];
```

```
  p[0] = 10;
```

```
  p[1] = 20;
```

//

p[0]	p[1]
10	20

```

delete [ ] P;
cout << "P[0]" << P[0] << endl;
cout << "P[1]" << P[1] << endl;
return 0;
}

```

Output : P[0] 22 → Garbage Value
P[1] 20

dynamic memory allocation, allocates memory.
during execution time.
Returns a pointer to the address where object is stored

Syntax pointer variable = new datatype
int *a = new int;
↳ allocates sufficient memory to hold the object of datatype int and returns a pointer to its starting point.
↳ Pointer variable holds the address of memory space all

int *a = new int; } can also be represented as
*a = 20;
int *a = new int(20);

memory deallocation

delete operator is used for releasing memory space when the object is no longer needed.

Syntax delete pointer variable;

↳ keyword.

↳ The pointer that points to the object already created in new operator

Imp points

- ① The programmer must take care not to free & delete a pointer that has already been deleted.
- ② The programmer must take care not to free or delete a pointer variable that have not being allocated using a new operator.
- ③ Overloading of new & delete operator is possible.
- ④ Null pointer is returned by the new operator when there is insufficient memory for allocation.

```
#include <iostream.h>
void main()
```

```
{
    int *a = new int;
```

```
    *a = 100;
```

```
    cout << "Output is " << *a;
```

// Value present in address location pointed by ptr var. *a is 100

```
    delete a; // memory is released using delete operator.
```

```
}
```

Functions

Simplest way to write function

include <iostream.h>

include <conio.h>

void add(int, int);

int main()

{

int a, b;

cout << "enter 2 no ";

cin >> a >> b;

add(a, b);

getch();

return 0;

void add (inta, intb)

{

int c;

c = a + b;

cout << "addition" << c;

}

ex. of W A N R

// func's prototype

function can be made

with

① N A N R

② W A N R

③ N A W R

R W A W R

N A b

w with

A Argument

R Return

funcⁿ called.
goes to its
definition.

// func's prototype is
written here

after execution goes back from
where it is called

Default Argument

work which system performs by itself is
called Default.

values on which work is to be performed
is argument. for ex in above example
a and b are arguments.

add (inta, intb, intc); // ^{prototype} Suppose you make this ⁽¹⁷⁾ funcⁿ
when this function will be called
we will call it like

add (5, 10, 15)
 ↑ ↑ ↑
 a b c

Default argument facilitates that if you have written a function and wants liberty to pass value or not then we use default argument

void add (inta , intb=5, intc=10); // ^{prototype}
now when you call this add function.

then passy value for a is mandatory

① add (5)

now if you want you can provide value for b and c or not ~~if~~

② can we pass variable

Yes

now when it will see that it has not provided any value for b & c then it will automatically provide 5 & 10 to be & c respectively as given in prototype.

③ now if you provide a value then it will ^{pass} ~~provide~~ that value which you ^{provide} ~~pass~~

if you dont provide a value then it takes ⁽¹⁸⁾
the default value given in prototype

suppose when we call funcⁿ add we
provide 2 values of a & b

add (5, 10)

so since value of c is not provided
then it will take default value from prototype
so it will be $a=5$

$b=10$

$c=10$

3) similarly if we provide all 3 values

add (5, 10, 15)

then values will be $a=5$

$b=10$

$c=15$

Ex

void add (int a=5, int b=10, int c=15)



add () // $a=5, b=10, c=15$

add (20) // $a=20, b=10, c=15$

add (10, 20) // $a=10, b=20, c=15$

add (10, 20, 30) // $a=10, b=20, c=30$

Rule if you start giving default arguments
in the variable then from there all the
variables after that should be provided

with default value

(14)

ex

void add (int a=5, int b=10, int c=15)
✓✓

void add (int a=5, int b, int c=15)
X

void add (int a, int b, int c=15)
✓✓

Default argument provides a way to give a default value to the arguments. If we don't supply the argument, it takes default value otherwise the passed value.

Rule Once we have started default value, all next argument must contain a default value.

Demo program,

```
#include <iostream.h>
#include <conio.h>
void add (int a=5, int b=10);
int main()
```

```
{
    add();
    add(10);
    add(10, 20);
    getch();
    return 0;
}
```

after funcⁿ call we go to funcⁿ definition.

```
void add (int a, b)
    {
        cout << "addition" << a + b;
    }
```

not required to write as written in prototype

Output

Addition = 15

Addition = 20

Addition = 30

// add()

// add(10)

// add(10, 20)

Inline functions

- Objectives of using functions in a program is to save memory space.
- when a function is called many times it gives advantages.
- Every time a function is called, it takes a lot of extra time in execution.
- This problem is solved using macros in C programming.
- But drawback of macros is no error checking.
- C++ solves this by using inline function.
- In inline function compiler replaces the function call with the corresponding funcⁿ code

Syntax inline function-header
{ function body
}

ex inline int sum (int a, int b)
{
 return a+b;
}

above inline funcⁿ can be invoked by statement like

C = sum(2, 5) // call C = 7

d = sum(4, 10) // call d = 14

Inline funcⁿ does not work
→ if loop, switch & goto exists
→ static variables.

funcⁿ is written to save memory space,
to reduce code redundancy

Suppose you write following add funcⁿ

```
void add (int a, int b)
```

```
{
```

```
-
```

```
}
```

So whenever we call a
function, a lot of work
is performed by compiler
internally.

Inline fun

Function is used to reduce code Redundancy
as well as to save our memory space.
When a funcⁿ is invoked lot of tasks is
performed viz matching arguments, matching
returns and passing the control from
calling to definition & vice versa.

But when function definition consist of
hardly one or two simple statements then the
bunch of tasks appears to be time consuming
Hence in order to save this time C++ has
the concept of inline function.

when a function is declared inline, the function body is replicated at function calling place.

Condition it must contain hardly one to two statements. no loops, switch & goto exist

To make a funcⁿ inline we use inline keyword before the function.

Demo program

```
#include <iostream.h>
#include <conio.h>
```

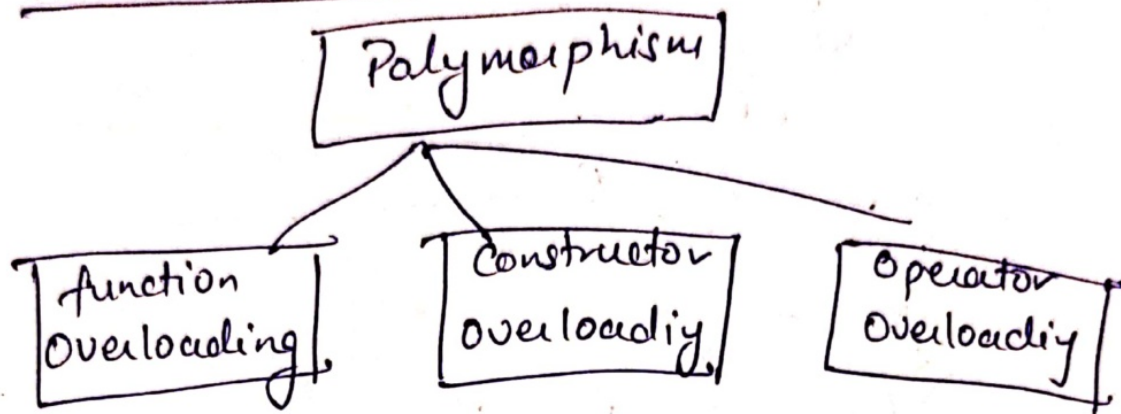
```
inline int square(int a)
{
    return a*a;
}
```

```
inline int cube(int a)
{
    return a*a*a;
}
```

```
int main()
{
    cout << "sq. of 5 = " << square(5);
    cout << "cube of 5 = " << cube(5);
    getch();
    return 0;
}
```

Function Overloading

24



Can two or more functions be written with same name in a C++ program?

Yes

Function overloading concept of polymorphism enables us to write same named multiple functions/methods within a program.

But we have a restriction that all the functions having same name must follow:

(i) There should be differences in

(a) no. of arguments

(b) Type of arguments

void add (int, int); } X not possible
void add (int, int); } because no. of arg are
same & type are also same
we can write them as

void add (int, int); } Possible
void add (int, float); } because type of
arg. is different

- ① void add (int, int);
- ② void add (int, float);
- ③ void add (int, int, int);

Why these differences should be there. because when compiler calls function with same name and same argument and data type then it will not be able to justify which funcⁿ to call.

```
void add (int, int);
void add (int, int);
```

} Compiler gets confused

```
void add (int, int);
void add (int, int, int);
```

} Compiler will be able to recognize

```
add(10, 20);
add(10, 20, 30);
```

```
#include <iostream.h>
#include <conio.h>
void add (int, int);
void add (int, int, int);
void add (int, float);
int main()
{
    add (2, 3);
    add (10, 20, 30);
    add (10, 20.10);
    getch();
    return 0;
```

```
void add (inta, intb)
{
    cout << "add " << a+b;
}
void add (inta, intb, intc)
{
    cout << "add " << a+b+c;
}
void add (inta, floatb)
{
    cout << "add " << a+b
```

OUTPUT

5
60
30.10

Constructors (made specially to initialize variables)

Value construction

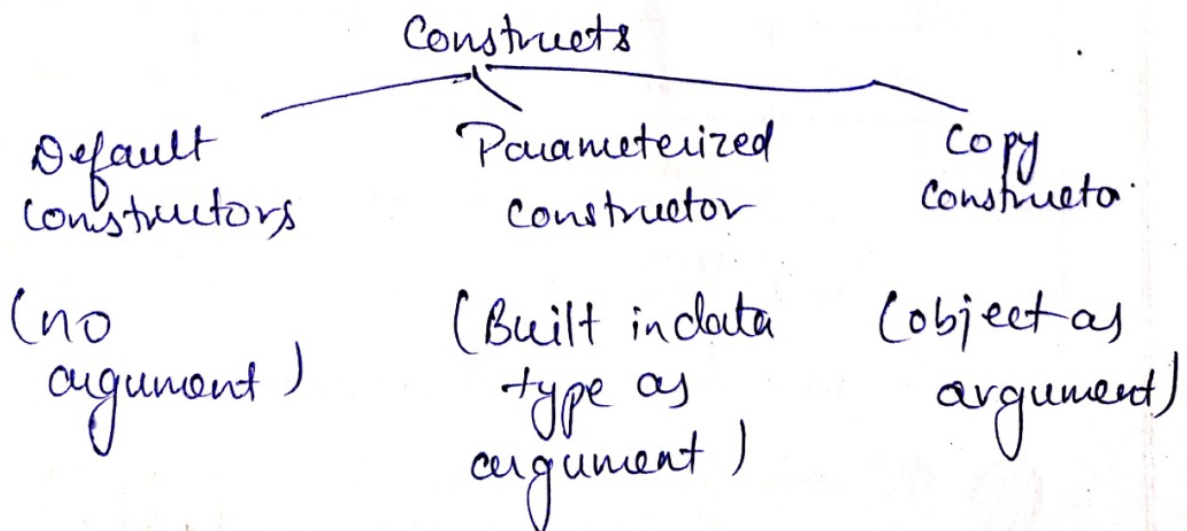
Constructors are special member function which is used to initialize value of a variable inside an object.

The major points about Constructors

- 1) A constructor's name is same as that of class name.
- 2) A constructor is automatically invoked as soon as an object of its class is created.
- 3) no return type in constructor not even "void".
- 4) Constructors allow default arguments concept.
- 5) Constructor cannot be inherited.
- 6) Constructors are always defined inside public section because object cannot access private section.
- 7) Constructors cannot be virtual.

Built in Datatype
int a=5;
making a variable a and also entering its value at same time

Class & object
use defined data type
abc aa=5;
we cannot write like this until now. To make it possible we use constructor,



① Default Constructors

is a type of constructors whose name is similar to the class name and which is automatically invoked. In ~~the~~ ^{The} ~~default~~ constructor having no argument is called default constructor.

Syntax

```
class abc
{
    int a;
    public :
        abc ( ) // Default constructor with no argument
        {
            a = 10;
        }
}
```

Defined inside class

can be defined outside class also

```
class abc
{
    int a;
    public :
        abc ( )
}

abc :: abc ( )
{
    a = 10;
}
```

Preferred inside class definition

outside class constructor definition

program

```
#include <iostream.h>
#include <conio.h>
```

class demo

```
{
    int a, b;
```

public:

demo()

```
{
    a = 10;
    b = 20;
```

```
}
```

void putdata();

```
};
```

void demo::putdata()

```
{
```

```
cout << "a = " << a << "\n b = " << b;
```

```
}
```

int main()

```
{
    demo aa;
```

```
    aa.putdata();
```

```
    return 0;
```

```
}
```

Constructor defined in class

// another function can be defined in or out of class

called constructor

aa object created

aa [a 10 b 20]

will go to putdata
a = 10 b = 20

OUTPUT

a = 10 b = 20

Parameterized Constructor

A constructor is a special member function whose name is same as class name and which is automatically invoked as soon as object of its class is created.

A constructor having argument is called parameterized constructor.

Syntax class demo

```
{ int a, b;  
public:  
    demo (int m, int n)  
    {  
        a = m;  
        b = n;  
    }  
}
```

~~demo~~ aa(10, 20);
demo aa();
X
error

aa { a
b }

```
#include <iostream.h>  
#include <conio.h>  
class demo
```

```
{ int a, b;  
public:  
    demo (int m, int n) // Constructor  
                        // with  
                        // Parameters  
    {  
        a = m;  
        b = n;  
    }  
}
```

```
void putdata()
```

```
{ cout << "a=" << a << "b=" << b;  
}
```

```
int main()  
{  
    demo aa(5, 10);  
    aa.putdata();  
    getch();  
    return 0;  
}
```

OUTPUT

a=5
b=10

Second way

(30)

```
int main ()
{
    int x, y;
    cout << "Enter 2 no: ";
    ✓ cin >> x >> y;
    demo aa (x, y);
    aa.putdata();

    getch();
    return 0;
}
```

} user input

OUTPUT
Enter 2 no 5 10
5 10

Copy Constructor

Constructors are special member functions when name is same as class name and which is automatically invoked when an object of its class is created.

~~Constructors~~ when we need to initialize the variable of an object with the values of variable of another object of same type (class), then we use ~~construct~~ concept of copy constructor.

example

how to
write
copy
constructor

```
demo aa;
demo bb = aa;
or
demo bb(aa);
```

// aa [a 5] b 10

now we want values of variables in aa should be copied in bb.


```
#include <iostream.h>
#include <conio.h>
```

```
class demo
```

```
{
```

```
int a;
```

```
Public:
```

```
demo ()
```

```
{
```

```
a=10;
```

```
}
```

```
demo (demo &z)
```

```
{
```

```
a=z.a;
```

```
}
```

```
void putdata()
```

```
{
```

```
cout << "\nA=" << a;
```

```
}
```

```
}
```

```
int main ()
```

```
{
```

```
demo aa;
```

```
demo bb(aa);
```

```
This line or
```

```
demo cc=bb;
```

```
aa.putdata();
```

```
bb.putdata();
```

```
cc.putdata();
```

```
getch();
```

★ Copy constructor (31)
should always
contain a default
constructor or a
parameterized constructor

// default
// constructor.

★ When we
make copy
constructor

then argument
address of
object of same class
is passed as an
argument for copy
constructor.

~~aa~~
aa

// copy
constructor

no new object is made
only address of object
made is kept.

A =

&z is address
of aa

aa[aa[10]]

// here we can pass aa as an argument.

all three Outputs will be
10

~~Definition in C++~~

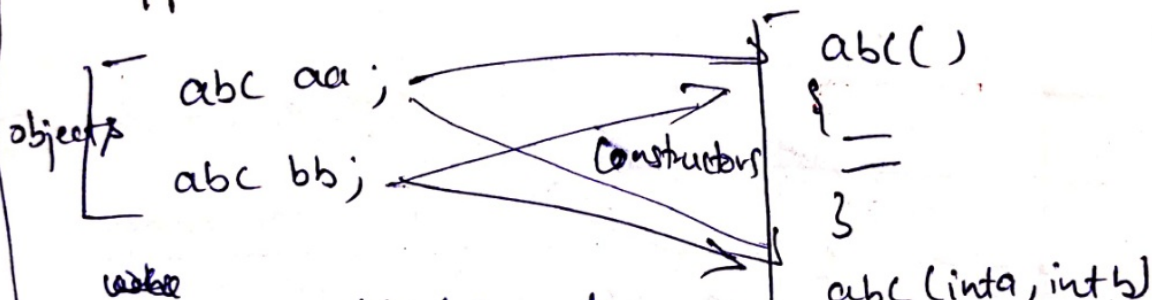
32

Constructor Overloading

We can have multiple constructors in a single program.

C++ provides us the provision using which we can implement or incorporate more than one constructor in a single program but these constructors must have different type of arguments or diff. no. of arguments.

Suppose we make 2 objects and 2 constructors



as soon as object is made constructor is called. So which constructor will be called as there are multiple constructor. To overcome this we use Constructor Overloading.

This provision of having more than one constructor in a single program is called constructor overloading.

What if we put same constructors suppose both constructors are default constructors then it will show error.

Which constructor will be called is dependent on no. of argument or type of argument.

Suppose

abc aa;
abc bb(5);

This will call this

```
abc ( )
{
  _
  _
}
```

It will call this

```
abc (int a)
{
  _
  _
}
```

Because of no. of argument
or
type of arguments

abc aa;
abc bb(5);
abc aa(5,6);

```
abc ( )
{
  _
  _
}
```

```
abc (int a)
{
  _
  _
}
```

```
abc (int a, float)
{
  _
  _
}
```

```
abc (int a, int b)
{
  _
  _
}
```

name of
Constructors could
be same but
no. of arguments or
type of argument
will be different

~~Constructors~~ :

```
#include <iostream.h>
#include <conio.h>
```

```
class demo
```

```
{ int a;
```

```
public:
```

```
demo ( ) // default constructor
```

```
{
  a=10;
}
```

```
demo (int x) // parameterized constructor
```

```
{
  a=x;
}
```

```
demo (demo &z) // copy constructor
```

```
{
  a=z.a;
}
```

```
void putdata ( )
```

```
{ cout << "\n A=" << a;
```

```
}
```

```
};
```

```
int main ( )
```

```
{ demo aa;
```

```
demo bb(20);
```

```
demo cc(aa);
```

```
aa.putdata();
```

```
bb.putdata();
```

```
cc.putdata();
```

```
getch();
```

```
return 0;
```

```
}
```

aa [a 10]

bb [a 20]

Destructor in C++

Destroys or destroys the values.

Constructors

- 1) values constructs
- 2) have arguments but no return type - not even void
- 3) Its name is similar to the class name

```
class A
{
    =
    A()
    {
    }
};
```

Destructors

- 1) Destroys the object
- 2) No argument, no return type
- 3) Its name is similar to the class name preceded by Tiled sign

```
class A
{
    =
    A() // Constructor
    {
    }
    ~A() // Destructors
    {
    }
};
```

Like Constructors destructors are also member functions whose name is similar to the class name having Tiled sign (~) just before the name.

A destructor is used to destroy the object once the object goes out of scope.

{ aa;

36

3
after this there is no value of aa then it will be destroyed and its memory will be released.

A destructor has no return type as well as it doesn't take any argument.

Example

#include <iostream.h>

~~class demo~~

int count = 0; // global variable

class demo

{ public:

demo()

{

count ++;

cout << "no. of object created" << count;

}

~demo()

{

count --;

cout << "no. of object destroyed" << count;

}

};

int main()

{

demo aa, bb, cc;

1
2
3
4

3. cc count++
3

count = 0
count++
1
count++
2

1. aa [1]
2. bb []




```

{
    demo del;
}
return 0;

```

after this scope is closed then all three objects will be destroyed in backward manner

// as soon as scope of del is closed automatically destructor will be called & del will be destroyed

Output

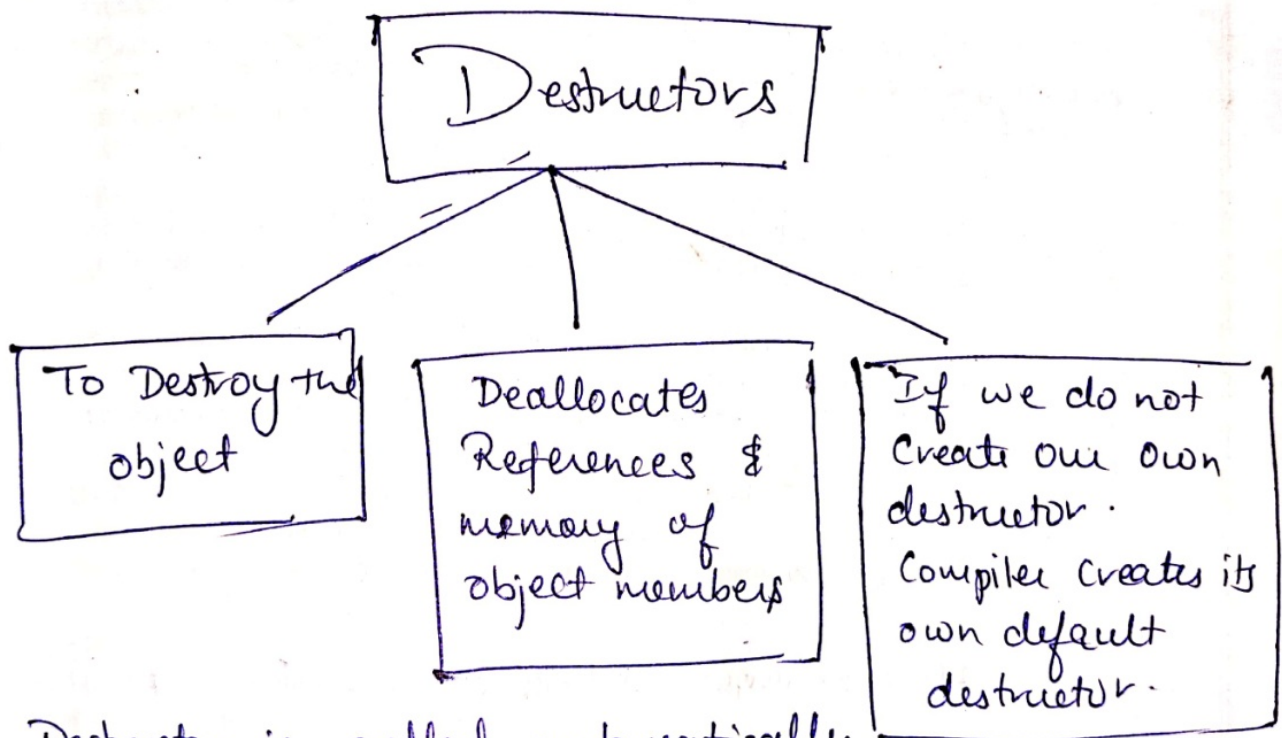
37

```

no. of object created 1
"no. of object created 2
"no. of object created 3
"no. of object created 4
"no. of object destroyed 3
"no. of object destroyed 2
"no. of object destroyed 1
"no. of object destroyed 0

```

first ce will be destroyed then bb & then aa



Destructor is called automatically when

- program finishes execution
- A scope ({} parenthesis) containing object ends.
- call the delete operator.

Friend Function

38

Object as a function argument

Program that adds 2 times

T1, T2 & T3 having both
hour section & minute section

T1	h [2]	m [40]
+		
T2	h [3]	m [50]
T3	h [6]	m [30]

void getdata (int, int)
if we want instead of
passing int, int as function
argument we can also
pass object as function
argument.

```
#include <iostream.h>
#include <conio.h>
```

```
class Time
```

```
{
    int h, m;
```

```
public:
```

```
void getdata()
```

// function 1

```
{
    cout << "Enter hours & minutes : ";
    cin >> h >> m;
```

```
}
```

```
void putdata()
```

// function 2

```
{
```

```
    cout << "hours = " << h << "Minutes = " << m;
```

```
}
```


void sum (Time T1, Time T2)

{ h = (T1.m + T2.m) / 60;

m = (T1.m + T2.m) % 60

h = h + (T1.h + T2.h)

}

// 40+50=90

$\frac{90}{60} = 1$ Quotient

20 Remainder

// 40+50=90 % 60
outputs 1 20

}

int main()

{ Time T1, T2, T3;

T1.getdata();

T2.getdata();

T3.sum(T1, T2);

// object T1 & T2 as sum
functions argument

T1.putdata();

T2.putdata();

T3.putdata();

getch();

return 0;

}

Friend function in C++

Suppose we have some circumstances that we have a function which is not inside the class but we want that funcⁿ to access the private section of that class. To make this possible, we will have to make that funcⁿ as a friend function.

Normal class

```
class abc  
{  
    int a, b;  
    public:
```

```
};  
abc aa;  
anything inside of a  
class can be used  
only by its object
```

We know that the private section of a class is accessible only and only through public section of same class.

What if we want to give access of private members to a function outside the class. In such circumstances we use the concept of friend function.

Key Points

- 1) A friend funcⁿ cannot be called using the object of the class. It is called like a normal function.
- 2) Friend funcⁿ can use the resource of a class only using an object of the same class.